

Projet d'Implémentation de méthodes Quantitatives élémentaires

Hugo Dunias

24 Novembre 2025

Table des matières

1	Introduction	2
2	Architecture du projet	3
3	Modèles de dynamique de prix (src/models)	4
3.1	Mouvement brownien géométrique (GBM)	4
3.2	Modèle de Heston	4
3.3	Modèle de Variance Gamma	5
4	Valorisation d'options (src/pricing)	6
4.1	Formule de Black–Scholes	6
4.2	Méthode de Monte Carlo	6
4.3	Solveur d'EDP (PDE) par différences finies	7
4.4	Greeks et sensibilités	7
5	Calibration des modèles (src/calibration)	8
5.1	Méthode des moments	8
5.2	Maximum de vraisemblance	8
5.3	Optimisation numérique	9
6	Mesures de risque (src/risk)	10
6.1	VaR et CVaR empiriques	10
7	Visualisation et notebooks	11
8	Conclusion	12

Chapitre 1

Introduction

Ce rapport présente l'architecture et les equations mathématiques d'un projet personnel informatique de familiarisation avec des implémentations de méthodes quantitatives élémentaires en python3.

Le code est organisé autour de quatre grands axes :

- la **modélisation stochastique** des prix d'actifs (`src/models/`),
- la **valorisation d'options** et le calcul de sensibilités (`src/pricing/`),
- la **calibration** des modèles (`src/calibration/`),
- la **mesure des risques** (`src/risk/`).

L'objectif de ce rapport est de faire le lien entre ces concepts théoriques et les modules effectifs du projet.

Chapitre 2

Architecture du projet

La structure logique du dépôt peut se résumer ainsi :

- `src/` :
 - `src/models/` : implémentation des modèles de dynamique de prix
 - `src/models/gbm.py`
 - `src/models/heston.py`
 - `src/models/variance_gamma.py`
 - `src/pricing/` : valorisation d'options
 - `src/pricing/black_scholes.py`
 - `src/pricing/greeks.py`
 - `src/pricing/pde_solver.py`
 - `src/pricing/monte_carlo.py`
 - `src/calibration/` : estimation des paramètres
 - `src/calibration/moments.py`
 - `src/calibration/likelihood.py`
 - `src/calibration/optimization.py`
 - `src/risk/` : mesures de risque (VaR et CVaR)
 - `src/risk/var.py`
- `notebooks/` : notebooks exploratoires de simulation et de visualisation ;
- `plotting.py` : utilitaires pour les graphiques et la visualisation de trajectoires.

Dans ce qui suit, nous décrivons les éléments mathématiques derrière chacun de ces blocs.

Chapitre 3

Modèles de dynamique de prix (src/models)

3.1 Mouvement brownien géométrique (GBM)

Le module `gbm.py` implémente le *Geometric Brownian Motion*, modèle canonique de Black–Scholes pour le prix d’un actif sans dividende. Sous la mesure risque-neutre $\mathbb{Q}$, la dynamique continutemporelle du prix $(S_t)_{t \geq 0}$ est donnée par l’EDS

$$dS_t = \mu S_t dt + \sigma S_t dW_t, \quad (3.1)$$

où W_t est un mouvement brownien standard, $\mu \in \mathbb{R}$ le drift et $\sigma > 0$ la volatilité.

Cette EDS admet une solution explicite :

$$S_t = S_0 \exp \left(\left(\mu - \frac{1}{2} \sigma^2 \right) t + \sigma W_t \right). \quad (3.2)$$

En particulier, $\log S_t$ est gaussien et les rendements logarithmiques sont normalement distribués.

Numériquement, la simulation se fait souvent par un schéma d’Euler sur la variable S_t :

$$S_{t_{k+1}} = S_{t_k} + \mu S_{t_k} \Delta t + \sigma S_{t_k} \sqrt{\Delta t} \xi_k, \quad (3.3)$$

où $(\xi_k)_k$ sont des variables i.i.d. $\mathcal{N}(0, 1)$. Le module `gbm.py` génère une matrice de trajectoires $(S_{t_k}^{(m)})_{m,k}$ à partir de ces incréments gaussiens.

3.2 Modèle de Heston

Le module `heston.py` implémente un modèle de volatilité stochastique. Dans le modèle de Heston, le prix S_t et la variance instantanée v_t vérifient

$$dS_t = \mu S_t dt + \sqrt{v_t} S_t dW_t^S, \quad (3.4)$$

$$dv_t = \kappa(\theta - v_t) dt + \sigma_v \sqrt{v_t} dW_t^v, \quad (3.5)$$

où

- $\kappa > 0$ est la vitesse de rappel vers le niveau de long terme $\theta > 0$,
- $\sigma_v > 0$ est la volatilité de la variance,
- les brownien W^S et W^v peuvent être corrélés avec corrélation ρ .

Numériquement, il faut simuler le processus pour v_t , dans le module `heston.py` ceci est implémenté via un schéma d’Euler. Une discrétisation simple s’écrit

$$v_{t_{k+1}} = \max(0, v_{t_k} + \kappa(\theta - v_{t_k})\Delta t + \sigma_v \sqrt{v_{t_k}} \sqrt{\Delta t} \xi_k^v), \quad (3.6)$$

puis, conditionnellement à v_{t_k} et $v_{t_{k+1}}$, on simule $S_{t_{k+1}}$.

3.3 Modèle de Variance Gamma

Le module `variance_gamma.py` implémente un modèle de processus de Lévy à sauts. Le log-prix est alors modélisé par

$$\log S_t = \log S_0 + rt + X_t, \quad (3.7)$$

où $(X_t)_{t \geq 0}$ est un processus de Variance Gamma, qui peut s'obtenir comme un mouvement brownien à drift θ et volatilité σ , *chronométré* par un processus gamma $(G_t)_{t \geq 0}$:

$$X_t = \theta G_t + \sigma W_{G_t}, \quad G_t \sim \Gamma\left(\frac{t}{\nu}, \nu\right), \quad (3.8)$$

où $\nu > 0$ contrôle la fréquence des sauts. Les incréments de X_t ont une distribution Variance Gamma $VG(\theta, \sigma, \nu)$, dont la fonction caractéristique est connue analytiquement.

Le module `variance_gamma.py` simule les incréments indépendants de G_t puis les mouvements browniens conditionnellement à ces temps aléatoires, afin de générer des trajectoires de log-prix à sauts.

Chapitre 4

Valorisation d'options (src/pricing)

4.1 Formule de Black–Scholes

Le module `black_scholes.py` implémente la formule de Black–Scholes pour le prix d'un call européen de maturité T et de strike K , dans le modèle `GBM.py` sous la mesure risque-neutre avec taux r et volatilité σ .

Le payoff est

$$\Phi(S_T) = (S_T - K)^+, \quad (4.1)$$

et le prix au temps 0 est donné par l'espérance actualisée

$$C_0 = e^{-rT} \mathbb{E}^{\mathbb{Q}}[(S_T - K)^+]. \quad (4.2)$$

En utilisant la distribution log-normale de S_T , on obtient l'expression fermée :

$$C_0 = S_0 N(d_1) - K e^{-rT} N(d_2), \quad (4.3)$$

$$d_1 = \frac{\log(S_0/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad (4.4)$$

$$d_2 = d_1 - \sigma\sqrt{T}, \quad (4.5)$$

où N désigne la fonction de répartition de $\mathcal{N}(0, 1)$.

De même, le prix du put s'obtient par parité put-call

$$P_0 = C_0 - S_0 + K e^{-rT}. \quad (4.6)$$

4.2 Méthode de Monte Carlo

Le module `monte_carlo.py` permet de valoriser des produits dérivés plus généraux en remplaçant l'espérance sous la mesure risque-neutre par une moyenne empirique sur des trajectoires simulées.

Pour un payoff $\Phi(S)$, le prix est

$$V_0 = e^{-rT} \mathbb{E}^{\mathbb{Q}}[\Phi(S)]. \quad (4.7)$$

En simulant M trajectoires $(S^{(m)})_{m=1, \dots, M}$, on approche cette espérance par

$$\hat{V}_0 = e^{-rT} \frac{1}{M} \sum_{m=1}^M \Phi(S^{(m)}). \quad (4.8)$$

Sous des hypothèses d'indépendance et de variance finie, le théorème central limite donne

$$\sqrt{M} \frac{\hat{V}_0 - V_0}{\hat{\sigma}} \xrightarrow[M \rightarrow \infty]{\mathcal{L}} \mathcal{N}(0, 1), \quad (4.9)$$

où $\hat{\sigma}^2$ est la variance empirique des termes de la somme. On pourrait ainsi également associer un intervalle de confiance à la valeur estimée.

4.3 Solveur d'EDP (PDE) par différences finies

Le module `pde_solver.py` implémente la résolution numérique de l'équation de Black–Scholes sous forme d'EDP par différences finies. Pour une option européenne de prix $V(t, S)$, l'EDP est

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0, \quad (4.10)$$

avec condition terminale $V(T, S) = \Phi(S)$.

On discrétise le temps et l'espace par des grilles

$$t_n = n\Delta t, \quad S_i = i\Delta S,$$

et on approxime les dérivées spatiales par des schémas de type

$$\frac{\partial V}{\partial S}(t_n, S_i) \approx \frac{V_{n,i+1} - V_{n,i-1}}{2\Delta S}, \quad (4.11)$$

$$\frac{\partial^2 V}{\partial S^2}(t_n, S_i) \approx \frac{V_{n,i+1} - 2V_{n,i} + V_{n,i-1}}{(\Delta S)^2}, \quad (4.12)$$

puis on intègre dans le temps avec un schéma de Crank–Nicolson (plus stable que les autres). Le code de `pde_solver.py` se ramène alors à la résolution itérative d'un système linéaire tridiagonal pour chaque pas de temps.

4.4 Greeks et sensibilités

Le module `greeks.py` calcule les sensibilités du prix d'option par rapport aux paramètres, en particulier :

- la **Delta** : $\Delta = \frac{\partial V}{\partial S_0}$,
- la **Gamma** : $\Gamma = \frac{\partial^2 V}{\partial S_0^2}$,
- la **Vega** : $\text{Vega} = \frac{\partial V}{\partial \sigma}$,
- la **Theta** : $\Theta = \frac{\partial V}{\partial T}$,
- le **Rho** : $\rho = \frac{\partial V}{\partial r}$.

Dans le cas Black–Scholes, on dispose de formules analytiques fermées. Par exemple,

$$\Delta_{\text{call}} = N(d_1), \quad (4.13)$$

$$\Gamma_{\text{call}} = \frac{N'(d_1)}{S_0 \sigma \sqrt{T}}, \quad (4.14)$$

où N' est la densité de la loi normale standard.

Dans le cadre Monte Carlo, ici on estime ces sensibilités par différences finies.

Chapitre 5

Calibration des modèles (src/calibration)

5.1 Méthode des moments

Le module `moments.py` se base sur l'idée de faire correspondre certains moments théoriques du modèle avec les moments empiriques observés sur les données de marché.

Soit R_t le rendement (logarithmique) du modèle. On peut, par exemple, considérer les moments

$$m_1(\theta) = \mathbb{E}_\theta[R_t], \quad m_2(\theta) = \mathbb{E}_\theta[R_t^2], \quad m_3(\theta) = \mathbb{E}_\theta[R_t^3],$$

où θ désigne le vecteur de paramètres du modèle (par exemple (μ, σ) pour GBM, ou (θ, σ, ν) pour Variance Gamma). Les moments empiriques $\hat{m}_k$ sont calculés à partir de la série temporelle observée.

La calibration par méthode des moments consiste à résoudre

$$\hat{\theta} = \arg \min_{\theta} \|m(\theta) - \hat{m}\|_W^2, \quad (5.1)$$

où W est une matrice de pondération. Ici W est l'identité et l'optimisation se fait avec le module `optimization.py`.

5.2 Maximum de vraisemblance

Le module `likelihood.py` implémente la maximisation de la vraisemblance. Si les rendements $(R_1, \dots, R_n)$ sont supposés i.i.d. sous une densité f_θ , la vraisemblance est

$$L(\theta) = \prod_{i=1}^n f_\theta(R_i), \quad \ell(\theta) = \log L(\theta) = \sum_{i=1}^n \log f_\theta(R_i). \quad (5.2)$$

Le maximum de vraisemblance est

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \ell(\theta). \quad (5.3)$$

Selon le modèle (GBM gaussien, Variance Gamma, etc.), la densité f_θ peut être explicite ou calculée numériquement via des transformées de Fourier. Le code évalue alors $\ell(\theta)$ pour un vecteur de paramètres donné, à partir des données de marché.

Ici aussi l'optimisation est faite avec le module `optimization.py`.

5.3 Optimisation numérique

Le module `optimization.py` fournit des routines d'optimisation pour résoudre les problèmes

$$\min_{\theta} J(\theta), \quad (5.4)$$

où $J(\theta)$ est soit un écart de moments, soit $-\ell(\theta)$ dans le cas du maximum de vraisemblance.

Les itérations suivent un schéma

$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \nabla J(\theta^{(k)}), \quad (5.5)$$

où ici la direction de descente est donnée par l'inverse approximatif de la matrice Hessienne.

$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k H_k \nabla J(\theta^{(k)}) \quad (5.6)$$

Chapitre 6

Mesures de risque (src/risk)

6.1 VaR et CVaR empiriques

Le module `var.py` implémente le calcul de la *Value at Risk* (VaR) et de la *Conditional Value at Risk* (CVaR) à partir d'une série de prix.

Étant donnée une série de prix $(P_t)_{t=0,\dots,T}$, on considère les rendements logarithmiques

$$R_t = \log P_t - \log P_{t-1}. \quad (6.1)$$

Pour un niveau de confiance $\alpha \in (0, 1)$, la VaR au niveau α est définie comme le quantile de niveau $1 - \alpha$:

$$\text{VaR}_\alpha = q_{1-\alpha}(R), \quad (6.2)$$

où $q_p(R)$ désigne le quantile d'ordre p de la distribution de R_t . Dans le code, ce quantile est estimé empiriquement à partir de l'échantillon $\{R_t\}_{t=1,\dots,T}$.

La CVaR est la perte moyenne conditionnellement à ce que la perte soit pire que la VaR. En termes de rendements,

$$\text{CVaR}_\alpha = \mathbb{E}[R_t \mid R_t \leq \text{VaR}_\alpha], \quad (6.3)$$

qui est approximée numériquement par la moyenne des rendements situés dans la queue de distribution au-delà du quantile.

Chapitre 7

Visualisation et notebooks

Le module `plotting.py` et le dossier `notebooks/` permettent de visualiser les trajectoires simulées, les distributions, etc.

D'un point de vue mathématique, ces visualisations illustrent :

- la distribution des trajectoires sous différents modèles (GBM vs Heston vs Variance Gamma) ;
- la convergence des estimateurs Monte Carlo (loi des grands nombres, TCL) ;
- l'effet de la calibration sur la forme des distributions et des surfaces de volatilité implicite ;
- la forme des queues de distribution et leur impact sur la VaR/CVaR.

Chapitre 8

Conclusion

Ce projet met en œuvre la chaîne complète de la finance quantitative moderne :

1. modélisation stochastique des prix via des EDS et des processus de Lévy ;
2. valorisation de produits dérivés, analytiquement (Black–Scholes) ou numériquement (Monte Carlo, EDP) ;
3. calcul de sensibilités pour la gestion de portefeuille et la couverture ;
4. calibration des modèles à partir de données de marché ;
5. mesures de risque basées sur les distributions empiriques des rendements.

Le rapport fournit la base théorique qui justifie les implémentations du code et est avant tout un support de compréhension et d'apprentissage personnel.